

0.48. Les réglages sous un nom : Carre

Carre ronde = { ... }; range les sept réglages sous un nom, en haut du programme ; carre(ronde); s'en sert.

1. Ce qu'il faut comprendre

Sept nombres à la suite se lisent mal : lequel est la taille, lequel la vitesse ? On les range donc **sous un nom**, en haut du programme, comme `Mot` au chapitre 1.

Ce qui est nouveau ici : le type `Carre`. `Carre ronde = { 10, 8, ALPHABET[0], 2, 1, 250, 2 };` range les sept réglages, **dans le même ordre** que `carre(...)`, sous le nom `ronde`.

`carre(ronde);` fait le carré avec ces réglages. Le compilateur remplace `carre(ronde)` par `carre(10, 8, ALPHABET[0], 2, 1, 250, 2)` : rien n'est rangé en mémoire, c'est gratuit.

L'avantage : pour changer le carré, on ne touche qu'à la ligne du haut, où chaque réglage a son titre au-dessus. Le programme, lui, ne dit plus que « fais le carré ronde ».

Il faut les sept réglages, ni plus ni moins. Il en manque un ? Le compilateur le dit, et donne un exemple complet.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2082 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Carre : un type pour ranger les sept réglages d'un carré sous UN nom,
// comme Mot range la place et le texte d'un mot (chapitre 1).
//
// Carre  ronde  = { 10, 8, ALPHABET[0], 2, 1, 250, 2 };
// |      |      |
// |      |      +-- les sept réglages, DANS LE MÊME ORDRE que carre(...),
// |      |      entre accolades { }, séparés par des virgules
// |      +----- le nom qu'on choisit : ronde
// +----- le type : Carre (avec une majuscule)
//
//          x  y  tuile      taille  sens  vitesse  tours
```

2. UNE VARIABLE — RONDE

```
Carre ronde = { 10, 8, ALPHABET[0], 2, 1, 250, 2 };
// autour de (10, 8), la lettre A, 5 × 5, sens des aiguilles, 250 ms par pas, 2 tours
```

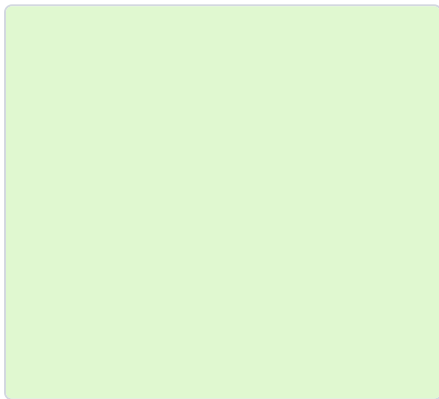
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // carre(ronde) veut dire : « fais le carré avec les réglages de ronde ».
    // Le compilateur la remplace par carre(10, 8, ALPHABET[0], 2, 1, 250, 2) :
    // c'est exactement le même programme, mais plus facile à lire.
    carre(ronde);

    // Pour changer le carré, on ne touche QUE la ligne de « ronde », en haut :
    // chaque réglage a son titre juste au-dessus.
    while (true) {
        image();          // attend l'image suivante (60 par seconde)
    }
}
```

3. Ce qui se passe, étape par étape

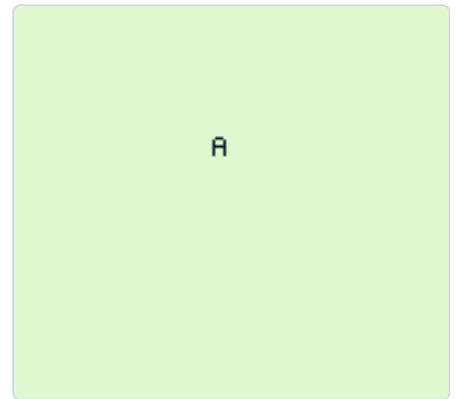
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



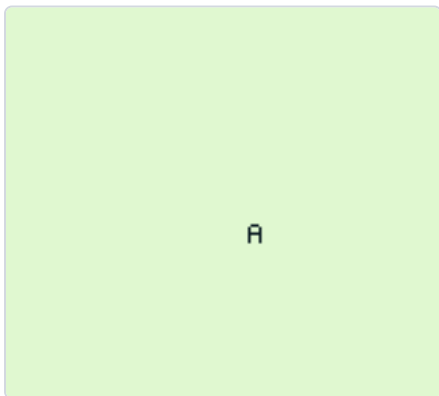
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui fait deux tours de 5×5 autour de (10, 8), et revient au centre.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d’être joués, et voici ce qu’ils ont mesuré.

- ✓ deux tours de 5×5 : deux passages au coin (12, 6) (2)
- ✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.48.1. Les réglages sous un nom : Carre — de base, ailleurs** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche
Game Boy en C++